DAY31-THREAT-MODEL

PART A — FUNDAMENTALS:

1. What is Insecure Design?

- Insecure Design occurs when an application lacks proper security controls during the design stage.
- It can create weaknesses that attackers may exploit.

2. Difference between insecure design and insecure implementation.

- Insecure Design: Security requirements and protections are missing or poorly planned.
- Insecure Implementation: Security requirements exist, but they are incorrectly implemented in the code.

3. What is Secure by Design?

- Secure by Design means considering security from the beginning of application development.
- Security controls are built into the system rather than added only after problems occur.

4. What is a security requirement?

- A security requirement defines what an application must do to protect its data and functionality.
- Examples include authentication, authorization, encryption and input validation.

5. What is an Asset?

- An asset is anything valuable that needs protection in a system.
- Examples include student records, passwords, marks and assignment files.

6. What is an Actor?

- An actor is a person, user or external system that interacts with an application.
- Examples include students, lecturers, administrators and attackers.

DAY31-THREAT-MODEL

7. What is an Entry Point?

- An entry point is a location through which users or external systems interact with an application.
- Examples include login forms, APIs, upload forms and password reset pages.

8. What is Attack Surface?

- Attack surface refers to all the possible points where an attacker could attempt to access or exploit a system.
- It includes application forms, APIs, network services and exposed endpoints.

9. What is a Trust Boundary?

- A trust boundary is a point where data or actions cross between different levels of trust.
- For example, when a student accesses an administrator-only function.

10. What is an Abuse Case?

- An abuse case describes how someone could misuse a legitimate application feature.
- For example, a student attempting to modify another student's marks.

11. Explain Least Privilege.

- Least Privilege means giving users and applications only the permissions required to perform their tasks.
- It reduces the damage caused by unauthorized access or compromised accounts.

12. Explain Defense in Depth.

- Defense in Depth uses multiple layers of security to protect an application.
- If one security control fails, other controls can still help protect the system.

13. Explain Fail Securely.

- Fail Securely means that when an application encounters an error or cannot verify permission, it should deny access safely.

- It should not expose sensitive information or grant unauthorized access.

14. Why should threat modelling happen early?

- Threat modelling helps identify potential security risks during the design stage.
- Fixing security weaknesses early is generally easier and less expensive than fixing them after deployment.

PART B — JAYVIK STUDENT PORTAL THREAT MODEL:

Analyze: Login, Student Dashboard, Admin Panel, Lessons, Assignment Upload, Assignment Review, Marks, Certificate Generation, Password Reset, API, Database, File Storage.

1. Identify at least 5 Assets

- Student personal information and profiles.
- Login credentials and tokens.
- Student marks and assessment feedback.
- Lessons and Learning materials.
- Database records.

2. Identify at least 4 Actors

- Student
- Lecturer
- Administrator
- Unauthenticated visitor

3. Identify at least 5 Entry Points

- Public Login form
- Password Reset
- Student Dashboard
- Admin Panel
- API endpoints.

4. Identify at least 3 Trust Boundaries

- Internet to Application
- Student to Administrator
- Application to File storage

DAY31-THREAT-MODEL

5. Identify at least 5 Threats / Abuse Cases

- Uploading malicious file or malware
- Uploading file with disguised extensions
- Accessing another student's uploaded assignment.
- Uploading overloaded file to exhaust storage.
- Modifying certificates.

Task 6: Recommend at least one appropriate security control for every identified threat.

- MFA aka Multi-factor authentication.
- File validation, Size limits and so on.
- Role based access control.
- MFA and Least Privilege

PART C — THREAT MODEL TABLE:

Create: Asset | Actor | Entry Point | Threat | Security Control (minimum 5 rows)

Asset	Actor	Entry Point	Threat	Security Control
Student accounts	Attacker	Login	Password guessing and account takeover	MFA, strong password hashing and rate limiting
Assignment files	Malicious student	Assignment Upload	Uploading malicious files	File validation, size limits and malware scanning
Student marks	Student	Dashboard / API	Accessing or modifying another student's marks	Role-based access control and authorization checks
Admin panel	Attacker	Admin Panel	Unauthorized administrative access	MFA and least privilege
Certificates	Malicious student	Certificate Generation	Forging or modifying certificates	Server-side generation and certificate verification
Password reset	Attacker	OTP endpoint	OTP guessing and account takeover	Short-lived OTPs and rate limiting
Database	Attacker	API	SQL injection and unauthorized data access	Parameterized queries and access controls

DAY31-THREAT-MODEL

PART D — DESIGN REVIEW:

Analyze Assignment Upload. Write security requirements for: Authentication, Authorization, File types, File size, Storage, Filename handling, Logging, Error handling.

Area	Security requirements
1. Authentication	Only registered and authenticated students should be allowed to upload assignments.
2. Authorization	Students must only upload assignments belonging to their own accounts and assigned modules. Lecturers should have appropriate review permissions.
3. File Types	Allow only approved file formats, such as PDF, DOCX and JPG. Validate file extensions and actual file types.
4. File Size	Set a maximum file size to prevent oversized uploads and storage exhaustion.
5. Storage	Store uploaded files securely outside the public web directory and prevent unauthorized access.
6. Filename Handling	Generate unique filenames on the server and reject unsafe filenames to prevent path traversal and file overwriting.
7. Logging	Record upload activities, including student ID, filename, upload time and upload status.
8. Error Handling	Display safe and clear error messages without exposing internal paths, server details or sensitive information.

PART E — BUSINESS LOGIC:

Analyze Certificate Generation. Secure flow should conceptually include: Student completion → Verification → Authorized approval → Certificate generation. Explain what could go wrong if the workflow can be bypassed.

1. Fake certificates: Students could generate certificates without completing the required lessons or assignments.

2. Unauthorized generation: Users could generate certificates without approval from an authorized lecturer or administrator.

3. Incorrect information: Attackers could modify student names, marks or completion details on certificates.

4. Privilege escalation: Students could access administrative certificate-generation functions without permission.

5. Duplicate certificates: Users could generate multiple certificates for the same achievement without proper checks.

PART F — REFLECTION:

Write 5-8 sentences: Why can perfectly written code still be insecure if the original application design is insecure?

Perfectly written code can still be insecure if the original application design does not include proper security controls. This is because security problems can exist in the system's architecture and workflow, it just not depends on the code. For example, an application may allow students to generate certificates without verifying their completion or obtaining approval. Even if the code works correctly, the design allows unauthorized certificate generation. Insecure design can also lead to weak access control, data exposure and privilege misuse. Developers must identify potential threats and security requirements before implementation. Therefore, security should be considered from the beginning of the development process through a Secure by Design approach.